ChatGPT

Designing an Agents-Teams-Native Opportunity Store

From undo tree to opportunity management

What you are describing is not best understood as “a better Git,” and not even as “a better undo tree.” It is a different control primitive. Git’s native model is an immutable object database of blobs, trees, commits, and tags, where each commit points to a single realized tree and to its parent commit or commits. Undo-tree systems, such as the well-known Emacs package, improve usability by preserving branching edit history instead of collapsing editing into a single linear redo chain, and Vim likewise exposes an undo tree rather than a purely linear undo log. But all of these systems still primarily track **realized past states**. They do not natively model **unrealized but plausible futures**, nor do they optimize over them as first-class objects. [1]

The architecture you are reaching for is closer to an **opportunity store** or **probabilistic solution-space archive**: a system in which a branch is not just “what happened,” but one member of a larger, evolving repertoire of candidate states, candidate plans, candidate agent teams, and candidate transformations. There is prior probability, posterior evidence, causal provenance, cost, novelty, risk, reversibility, and utility attached to each candidate. Research on uncertain version control already points in this direction: one early model represented versions as graph derivations with probabilistic events, while work on multi-version typed graphs shows that many versions can be stored compactly inside one graph and reasoned about before pairwise merges even happen. [2]

That shift matters because current software and AI workflows increasingly generate code, content, and even candidate workflows cheaply, while compute, memory, distribution, and human attention remain scarce. In that setting, the central problem is less “how do I preserve history?” and more “how do I manage competition among possibilities under constraints?” Recent work on self-evolving agents makes the same diagnosis from the AI side: the field is moving from static models toward agents that evolve prompts, workflows, memory, tools, and even internal architecture over time. [3]

I would therefore define the core idea as follows: **replace version control over realized states with governed exploration over a probabilistic space of opportunities**. “Undo” becomes only one operator inside a larger family of reversible transformations; “branch” becomes one local view of a broader repertoire; and “merge” becomes only one kind of updating operation, alongside selection, recombination, compression, projection, and Bayesian revision. That is the conceptual move that makes your “CMD-Z + tree + store/market” metaphor coherent rather than poetic only. This interpretation is consistent with work on local-first systems, event-sourced agents, quality-diversity search, and probabilistic reasoning over graph-structured state. [4]

A formal model for the system

The cleanest formal foundation for this architecture is a combination of **event sourcing**, **typed graphs**, **semiring provenance**, **probabilistic circuits**, and **quality-diversity archives**. Event sourcing gives you an append-only causal log and replayable materialized views; typed graph models let many compatible or competing versions coexist in one compact structure; semiring provenance gives algebraic bookkeeping for why a derived fact or state exists; and probabilistic circuits such as PSDDs give a tractable way to represent distributions that must also satisfy logical constraints. [5]

In practical terms, the system should treat the following as first-class entities. A **mutation** is a proposed transformation produced by a human, an agent, or an algorithm. A **state** is not just a file tree but a typed graph containing data, code, notes, tests, policies, and relations among them. A **projection** is a computable view of that graph for one context, such as a runnable build, a note sequence, a dashboard, or a market-style opportunity map. A **witness** is the provenance explaining why a state or score exists. A **belief** is a probability distribution over candidate states, outcomes, or causal effects. An **archive cell** is one niche in behavior space, holding the current elite or representative for that niche. Those concepts are not arbitrary; they follow naturally from the literature on provenance semirings, probabilistic version models, PSDDs, and MAP-Elites-style repertoires. [6]

If you want category theory to do real work here, use it to define the **schema and composition semantics**, not as decorative vocabulary. Catlab.jl is explicitly built for applied and computational category theory and supports wiring diagrams and algebraic structures suitable for compositional system design; Sage treats categories as a software-engineering pattern for organizing generic code; and GAP and Sage provide mature computational backends for groups and related algebraic objects. That makes category and group theory excellent for formalizing composition rules, symmetries, and invariants, but they should sit in a research/formalization lane rather than the hot path of the runtime. [7]

A useful internal slogan is this: **the runtime stores possibilities, the algebra defines legal composition, the probabilities rank belief, and the archive preserves diversity**. That makes the system more like a governed ecological-evolutionary substrate than a source-control repository. Mean-field and game-theoretic views of multi-agent systems are a natural fit because they give a language for resource contention, cooperation, equilibrium, and population-level dynamics; recent surveys and causal mean-field MARL work show why population structure and causal aggregation matter when many agents co-adapt under limited communication and shared constraints. [8]

The application and library stack I would actually build

I would build this as a **Rust-first core with a Python research lane and a TypeScript desktop shell**. Rust is a strong fit because its ownership and type systems were explicitly developed to make reliability and concurrency bugs easier to manage, and the language positions itself around reliable and efficient software. For a system that is meant to be radically innovative **and** controlled, this matters more than syntactic convenience. PyO3 and Maturin then give a well-supported bridge from Rust into Python packages and mixed Rust/Python development. [9]

For the **core data model**, use Rust with `petgraph` for general graph structures and custom traversal logic, and use GraphBLAS where sparse matrix and semiring formulations are the right abstraction.

`petgraph` gives multiple graph types plus algorithms and Graphviz export in Rust. GraphBLAS, by design, defines generalized matrix and vector operations over semirings that can express a wide range of graph algorithms, which is exactly what you want if graph traversal, sparse tensor algebra, and provenance propagation need to unify rather than live in separate subsystems. [10]

For the **storage substrate**, I would separate operational state from analytical state. Operationally, use an append-only event log and a persistent embedded store such as RocksDB; analytically, use Arrow-compatible columnar structures with Parquet-backed snapshots, queried through DuckDB and Polars. RocksDB is an embeddable persistent key-value store with a log-structured engine oriented toward fast storage. Apache Arrow gives a language-independent in-memory columnar format with zero-copy reads and a stable format. DuckDB is an in-process OLAP engine built on a fast columnar storage engine, while Polars' lazy API enables whole-query optimization and parallelism. Together, those choices support both event replay and large-scale telemetry analysis without turning the runtime into a database product. [11]

For **compression and compact representations**, use Zstandard for snapshots, deltas, caches, and replicated transport. Zstandard is a fast lossless compression algorithm with a broad speed/compression trade-off and support for dictionary compression, which is useful when your store repeatedly handles structurally similar artifacts such as prompts, traces, graphs, note cards, and model metadata. For the probabilistic reasoning layer, use compact symbolic structures such as probabilistic circuits or extended decision diagrams where exact or bounded inference is needed. Recent work on PSDDs and typed extended decision diagrams shows why symbolic compression matters when logically constrained probabilistic state gets large. [12]

For the **local-first collaborative layer**, use Automerge rather than inventing your own sync semantics in version one. Automerge describes itself as a local-first sync engine for multiplayer apps that works offline, prevents conflicts, and uses CRDTs; Automerge Repo adds pluggable storage and network adapters and treats documents and repositories as the main abstraction boundary. That aligns almost perfectly with your "store" metaphor, especially if notes, workflows, hypotheses, and experiment cards should all be editable offline and sync without an always-authoritative central server. Local-first principles more broadly support user control, offline work, long-term preservation, and collaboration without making the cloud the owner of the truth. [13]

For the **research and modeling lane**, I would be deliberately dual-track. Use PyTorch as the default numeric substrate for the general system because it offers an optimized tensor library, autograd, and distributed primitives, and pair it with PyTorch Geometric for graph ML. Keep JAX plus NumPyro as a focused experimental lane for probabilistic surrogates, calibration, and fast transformation-heavy models, because JAX is built around program transformations and JIT compilation, while NumPyro is explicitly a lightweight probabilistic programming library built on JAX. For richer Bayesian/deep probabilistic work, Pyro remains a very good complement on the PyTorch side. [14]

For **graph analytics and causal inference**, use NetworkX for rapid prototyping and algorithmic experimentation, then escalate to GraphBLAS- or PyG-backed pathways when performance matters. NetworkX remains excellent for manipulating and studying graph structure, while DoWhy and EconML provide explicit causal graphs, refutation machinery, heterogeneous treatment-effect estimators, and interpretability tools that are directly relevant if the system is supposed to maintain predictive power rather than just correlations. This is important because your architecture is not merely generating options; it is trying to understand which transformations actually cause useful changes in outcomes. [15]

For the **agent runtime and control plane**, combine Ray with Temporal rather than asking one tool to do both throughput and governance. Ray provides small, composable distributed primitives—tasks, actors, objects, and an object store—which are excellent for search, simulation, and batched evaluation. Temporal is explicitly a durable execution platform whose workflow executions resume from recorded event history after crashes or outages. In your architecture, Ray should do high-throughput exploration, while Temporal should own long-running governed workflows, replay, and guaranteed completion semantics. [16]

For the **desktop application shell and embodied interface**, build with Tauri plus a web frontend. Tauri gives you a Rust core with a web UI and can target the major desktop platforms from one codebase. Use React Flow for the opportunity graph editor because it is purpose-built for node-based editors and interactive diagrams with customizable nodes and edges. Use Motion for animated transitions, density shifts, and reversible "CMD-Z-like" movement cues. If you later want GPU-native visual fields, use `wgpu`, which is a safe and portable Rust graphics library based on WebGPU and suitable for both graphics and compute. [17]

For **observability, governance, and constraints**, use OpenTelemetry for traces, metrics, and logs; Prometheus for operational query and alerting; and OPA for declarative policy gates. OpenTelemetry gives a standard framework for generating and exporting telemetry, including spans and context propagation; Prometheus provides real-time time-series querying through PromQL; and OPA's Rego is designed specifically for policy evaluation over structured data. In your system, every mutation, evaluation, acceptance decision, and replay should emit traceable causal evidence, and every promotion of a candidate should pass policy as code. [18]

The crucial algorithms and why RLHF should not be your organizing principle

If the core object is a **space of possibilities**, then the organizing algorithm should not be "optimize one policy against one scalar reward and sometimes use human preferences." It should be **multi-objective search over a structured repertoire**. MAP-Elites is foundational here because it stores the highest-performing solution in each niche of a behavior space, and differentiable and Bayesian variants show how to make that process faster and more sample-efficient when gradients or surrogate models exist. This is a much better fit for your concept of recurrent search under changing constraints than a single-reward hill climb. [19]

That archive should be fed by a mix of **mutation operators**. Some mutations will be symbolic and compiler-like: graph rewrites, schema rewrites, policy rewrites, prompt rewrites, note-link rewrites, or agent-team topology rewrites. Some will be neural: latent-space perturbations, policy edits, or embedding shifts. Recent work on LLMs as evolutionary optimizers and on structured debate-driven prompt evolution shows that language models can be used as variation operators inside evolutionary frameworks rather than only as policies trained through RLHF. The self-evolving agents literature suggests that what evolves may include prompts, workflows, memory, skills, tools, and architectures—not only model weights. [20]

Human preference should remain one signal, but not the constitution of the system. DPO was influential precisely because it showed that RLHF's goal can often be approached with a simpler and more stable preference-learning formulation, which already weakens the claim that "reinforcement learning with human feedback" must be the universal alignment substrate. At the same time, active inference provides a different

control-theoretic perspective in which agents act to reduce expected surprise under a generative model, not merely to maximize an externally specified reward. For your architecture, the implication is straightforward: preferences should be one axis inside a broader decision surface that also includes correctness, novelty, robustness, causal effect, cost, reversibility, ecological fit, and policy compliance. [21]

To maintain predictive power, the system also needs **interpretable internal instrumentation**. Compiler-based interpretability work such as Tracr and ALTA supports the idea of representing agent programs as typed intermediate representations that can be compiled, inspected, and tested. Causal tracing methods and sparse autoencoders support a second layer of observability inside the models or controllers themselves, helping localize which components matter for outcomes and providing more faithful debugging signals than end-task performance alone. That is highly aligned with your processor-tracing and epistemology themes: not only "what solution won," but "what internal mechanism created that outcome?" [22]

The probabilistic reasoning layer should use **symbolic compression where structure matters**. PSDDs are valuable because they compactly represent probability distributions while respecting logical constraints. Provenance semirings are valuable because they preserve algebraic explanations of how derived artifacts depend on source facts or events. Recent work on layered provenance and probabilistic query reasoning strengthens the case that compact symbolic lineage is not an academic luxury; it is one of the few ways to keep exact or bounded inference tractable as the solution space becomes combinatorially large. [23]

Finally, compute should be allocated with explicit regard to **scaling laws and inference demand**. Kaplan-style and Chinchilla-style results show that model performance obeys predictable power-law structure with respect to parameters, data, and compute, and later work that accounts for inference demand shows that deployment requirements can change the compute-optimal operating point. In your system, that means agent populations, surrogate models, and archive sizes should be scaled as **budget-aware design variables**, not as prestige metrics. Sustainability is not a moral add-on here; it is a direct architectural consequence of the economics of scaling. [24]

Reliability, control, and predictive power

A self-evolving architecture fails if its acceptor is weak, even when its proposer is brilliant. Recent work on PACE argues exactly this point: naive "keep it if dev score went up" acceptance rules induce uncontrolled adaptive testing and spurious self-improvement. That critique is extremely relevant to your concept because an opportunity store will naturally produce many candidate mutations. Without a statistically disciplined commit gate, you convert exploration into drift. [25]

I would therefore make **acceptance** a governed pipeline with four layers. The first layer is **hard validity**: types, schema invariants, graph constraints, policy rules, and deterministically checkable properties. The second is **task evidence**: paired evaluation against the incumbent with anytime-valid or sequential testing. The third is **causal and interpretability evidence**: did the change alter the causal drivers you expected, or did it merely overfit a benchmark? The fourth is **archive logic**: even if the candidate is not globally best, does it improve coverage of a niche, reduce risk, or provide a reversible option under a different constraint regime? Those choices are supported by current work on anytime-valid agent acceptance, formal hard constraints for self-evolving agents, causal inference tooling, and QD archives. [26]

This is also why I would not frame “undo” as simple rollback. In an event-sourced, local-first, probabilistic architecture, the correct mental model is **reprojection**. You do not merely jump back to a previous point; you project a different world-state from the same causal substrate under a different selection of events, policies, and constraints. Temporal’s event history, event-sourced agent architectures, and local-first CRDT systems all support versions of this logic already: durable logs and replay are the substrate, and materialized current state is just one view. [27]

The observability model should mirror that. Every mutation should emit spans, scores, causal graph changes, resource costs, and compression deltas. Every promoted candidate should carry its provenance witness, policy verdict, evaluation seeds, replay handle, and archive niche descriptor. OpenTelemetry spans and propagated context, plus Prometheus metrics, give you the operational substrate for that. The result is a system that can say not only “this won,” but also “under these constraints, with this evidence, at this cost, and by these mechanisms, this was accepted.” That is the minimum bar for “maintain control and predictive power” in a self-evolving architecture. [28]

A pragmatic roadmap

The first product should be a **single-user local-first desktop application** with a strong library core. Make the app a Tauri desktop shell over a Rust kernel and a React Flow canvas. In this phase, do not try to solve full autonomous evolution. Instead, build the opportunity graph, typed event log, archive cells, projections, note cards, constraint overlays, and replay/reprojection mechanics. Automerge can already provide offline-first replicated document semantics, and React Flow plus Motion can make the “CMD-Z as landscape navigation” metaphor tangible in a way that users can feel rather than merely read about. [29]

The second product should be a **research library** that exposes the formal model cleanly to Python. The Rust side should own storage, graph kernels, provenance, compression, and replay. The Python side should expose experiment APIs for mutation operators, surrogate models, causal analysis, graph ML, and archive evaluation. PyO3 and Maturin make this packaging strategy practical, while PyTorch, PyG, JAX, NumPyro, and DoWhy/EconML cover most of the numerical and causal substrate you are likely to need without prematurely fragmenting the stack. [30]

The third phase should introduce **agents-teams-native execution**. Here, the primary abstraction should not be a single assistant or a static workflow DAG. It should be a typed team graph in which agent roles, memory scopes, tools, and constraints are mutable and subject to evaluation. Ray should handle distributed search and simulation over many candidate team configurations, while Temporal should own the durable long-running workflows that matter to users. This is where quality-diversity becomes central: you want a repertoire of viable team organizations, not only one “best” topology. [31]

The fourth phase is where category theory and algebra should move from sidecar to advantage. Use Catlab and algebraic rewriting ideas to formalize graph rewrite rules, composition operators, and semantic invariants for agent teams, workflows, and note structures. Use Sage and GAP only where exact algebraic computation creates real leverage, such as discovering symmetries, quotienting equivalent structures, or defining canonical forms for certain transformations. This keeps the production system elegant without letting the formal methods layer dominate the runtime budget. [32]

If I were naming the architecture internally, I would choose something like **Opportunity Store**, **Z3 Store**, or **Repertoire Store**. The important point is not the branding, but the conceptual contract: this is a system for

storing, compressing, evaluating, and navigating **possibility under constraint**. The strongest version of your idea is not anti-version-control and not anti-learning-from-humans. It is a deeper substrate in which versioning, preferences, causality, search, economics, and adaptation become coordinated views of one underlying store. That is a credible and genuinely novel research direction, and the current literature is mature enough to support a serious first implementation. [33]

---

[1] Git - gitdatamodel Documentation
https://git-scm.com/docs/gitdatamodel?utm_source=chatgpt.com

[2] [6] [33] Towards a Version Control Model with Uncertain Data
https://pierre.senellart.com/publications/ba2011version.pdf?utm_source=chatgpt.com

[3] A Survey of Self-Evolving Agents: On Path to Artificial Super Intelligence
https://arxiv.org/abs/2507.21046?utm_source=chatgpt.com

[4] Local-First Software:You Own Your Data, in spite of the Cloud
https://martin.kleppmann.com/papers/local-first.pdf?utm_source=chatgpt.com

[5] ESAA: Event Sourcing for Autonomous Agents in LLM- ...
https://arxiv.org/pdf/2602.23193?utm_source=chatgpt.com

[7] [32] Catlab.jl
https://algebraicjulia.github.io/Catlab.jl/dev/?utm_source=chatgpt.com

[8] A Survey of Multi Agent Reinforcement Learning
https://arxiv.org/pdf/2507.06278?utm_source=chatgpt.com

[9] Rust Programming Language
https://rust-lang.org/?utm_source=chatgpt.com

[10] petgraph - Rust
https://docs.rs/petgraph/?utm_source=chatgpt.com

[11] RocksDB | A persistent key-value store | RocksDB
https://rocksdb.org/?utm_source=chatgpt.com

[12] Zstandard - Real-time data compression algorithm
https://facebook.github.io/zstd/?utm_source=chatgpt.com

[13] llms.txt
https://automerge.org/llms.txt?utm_source=chatgpt.com

[14] PyTorch documentation — PyTorch 2.12 documentation
https://docs.pytorch.org/docs/stable/index.html?utm_source=chatgpt.com

[15] NetworkX — NetworkX documentation
https://networkx.org/?utm_source=chatgpt.com

[16] [31] What's Ray Core? — Ray 2.55.1 - Ray Docs
https://docs.ray.io/en/latest/ray-core/walkthrough.html?utm_source=chatgpt.com

[17] [29] Tauri 2.0 | Tauri
https://v2.tauri.app/?utm_source=chatgpt.com

18 Documentation
https://opentelemetry.io/docs/?utm_source=chatgpt.com

19 Quality Diversity Algorithms
https://members.loria.fr/jbmouret/qd.html?utm_source=chatgpt.com

20 Large Language Models as Evolutionary Optimizers
https://arxiv.org/html/2310.19046v3?utm_source=chatgpt.com

21 Direct Preference Optimization: Your Language Model is Secretly a Reward Model
https://arxiv.org/abs/2305.18290?utm_source=chatgpt.com

22 Compiled Transformers as a Laboratory for Interpretability
https://openreview.net/forum?id=tbbId8u7nP&utm_source=chatgpt.com

23 Probabilistic Sentential Decision Diagrams
https://starai.cs.ucla.edu/papers/KisaKR14.pdf?utm_source=chatgpt.com

24 Scaling Laws for Neural Language Models
https://arxiv.org/abs/2001.08361?utm_source=chatgpt.com

25 26 PACE: Anytime-Valid Acceptance Tests for Self-Evolving Agents
https://arxiv.org/abs/2606.08106?utm_source=chatgpt.com

27 What is Temporal? | Temporal Platform Documentation
https://docs.temporal.io/temporal?utm_source=chatgpt.com

28 Traces
https://opentelemetry.io/docs/concepts/signals/traces/?utm_source=chatgpt.com

30 Introduction - PyO3 user guide
https://pyo3.rs/?utm_source=chatgpt.com